

The Flight Challenge: how to compete

Session #3 — Advanced, part 2: scripts, flags, submissions, and the board

Tutorial track

RL Bootcamp 2026 · Salzburg, 16–18 September

Scoring v2.0.0 — `sessions/03-advanced/competition/`; every command here is in `docs/advanced/kaggle.md` and `competition.md`

Two deliveries, three numbers

What you hand in, where, and when

🕒 25 min

	Kaggle — the submission	The zip — on request, and at the close
What	submission.csv: your agent's action at every (seed, step)	submission.yaml + model/policy.zip + every file you changed
Where	the competition page (join link on slide 5)	the shared Drive folder, as <team>.zip
When	all session, as often as you like (20 a day)	once , by the sprint deadline; re-upload to revise
Scored on	the 100 <i>published</i> seeds, by Kaggle	100 <i>secret</i> seeds, by us, offline, one zip per process

⚠ Kaggle is how you submit. The zip is how we check.

If the top of the board looks **close**, we will ask the teams involved for their **code and zips** and score them ourselves on seeds nobody has seen. Keep the zip valid all afternoon.

💡 Submit the rule-based agent first

Two minutes, a row with your name on the board, and your pipeline proven end to end before anything you trained is ready.

Three numbers you will see — and only one decides the trophies

	What it scores	When you see it	Secret?
Kaggle public	60 of the 100 <i>published</i> seeds	live, every submission	no
Kaggle private	the other 40 <i>published</i> seeds	when the competition closes	no
Ours	100 seeds you have never seen	at the ceremony	yes

Why Kaggle's hidden half is not actually secret

A submission is an **action trace keyed by seed**. To write the rows for a seed you must roll out on it, so every seed Kaggle scores is in `competition/seeds_public.txt`, committed. Its split hides 40 seeds from *view* until the close, not from *you*.

Expect all three to disagree

The published seeds are published, so you will tune against them, and that is fine. The gap between your Kaggle score and the ceremony score is the difference between a policy that solved the sector and one that memorised a hundred scenarios.

The public board

1 — Build the file

>_ From sessions/03-advanced, with rlbootcamp active

```
cd sessions/03-advanced
python -m competition.run submit --agent rule-based --out submission.csv
python -m competition.run submit --agent airtraffic/runs/mine/final_model.zip --out
submission.csv
```

>_ **stdout** — what **submit** prints: your **local** score on the public seeds, rank key in board order

```
{
  "failed_episodes": 10,
  "congestion_total": 176,
  "exit_miss_mean": 0.1288,
  "clearances_total": 2089,
  "aircraft_involved_total": 477,
  "rank_key": [10, 176, 26,
2089, 477]
}
```

--agent: noop, rule-based, or any SB3 .zip — rolled out on the **stock** environment. Changed yours?
ingest --out, two slides on.

 **submission.csv** — the file you upload: a header, then 100 seeds $\times$ 50 steps = 5 001 lines

```
id,action
s12146695_t00,1
s12146695_t01,1
...
s12146695_t49,0
```

Every seed carries all 50 rows

An episode that ends early is padded with the idle action — Kaggle's solution file has 50 rows per seed and rejects anything else. submit pads for you; --limit files are **not** uploadable.

Every flag in the harness

competition.run — global flags go *before* the subcommand

<code>--limit N</code>	the first <i>N</i> public seeds: a quick check, not a valid upload
<code>--stochastic (submit, compare)</code>	sample actions instead of the argmax — what <i>Iron Stomach</i> measures
<code>--seeds FILE</code>	an explicit seed file
<code>--n-lo N / --n-hi N</code>	narrow the traffic draw: development only , every line stamped off-board
<code>-v / --private</code>	per-seed progress / the secret list — organisers, the file is not in the repo
<code>submit --agent A --out F</code>	A = noop random rule-based an SB3 .zip, rolled out on the stock env
<code>replay --submission F [--strict]</code>	score a file exactly as Kaggle will; <code>--strict</code> also rejects extra seeds
<code>compare --agent A [--agent B ...]</code>	one ranked table, the board's rank key
<code>postmortem --agent A</code>	every diagnostic pass; the trophies when more than one agent
<code>...[--extraction-k 8]</code>	
<code>decode SCORE</code>	a leaderboard number back into its four KPIs
<code>score-zips --dir D [--no-contract]</code>	the private board — organisers

competition.ingest — one zip: your dry run, and your route to Kaggle if you changed the env

<code>--zip Z [--contract]</code>	what the private board does to your zip, on the public seeds; the contract suite first
<code>--out F</code>	write the Kaggle trace from a policy that observes <i>your</i> environment
<code>--limit N / --workdir D</code>	first <i>N</i> seeds / keep the unpacked files where you can read them

2 — Check it yourself, then 3 — upload it

>_ The same replay Kaggle runs

```
python -m competition.run replay --submission submission.csv
```

⚠ Step 2 is not optional

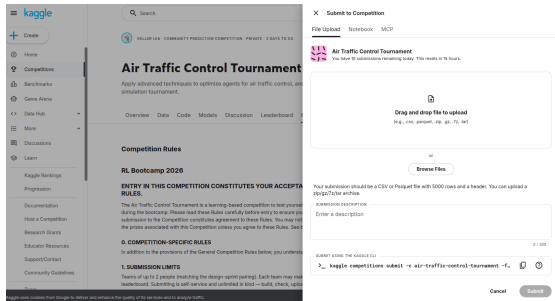
If `replay` disagrees with what `submit` printed, the problem is your *file*, not your agent — a stale checkpoint or a different seed list. Find out here, not on the board.

Upload

Submit Prediction → drop `submission.csv` → **describe it** — “relative obs, 2M steps” beats “submission 7” three hours later → **Submit**.

Join (unlisted; the link is the only way in):

`kaggle.com/t/a0a74b5818114812a2b076b172dfe197`



The *Submit Prediction* drop zone, with a description that says what the run was

Reading the number: lower is better, and only the order means anything

Leaderboard

[Raw Data](#) [Refresh](#)[Public](#) [Private](#)

This leaderboard is calculated with approximately 80% of the test data. The final results will be based on the other 20%, so the final standings may be different.

#	Team	Members	Score	Entries	Last	Join
1	submission_rulebased.csv		52022965198815...	1	12d	
2	submission_noop.csv		240116496344596...	1	12d	

The two reference rows, seeded before the room opens

agent	Kaggle public	what it is
PPO, 4M steps	1.20×10^{13}	the shipped checkpoint
rule-based	3.60×10^{13}	the bar
noop	2.40×10^{14}	the brick

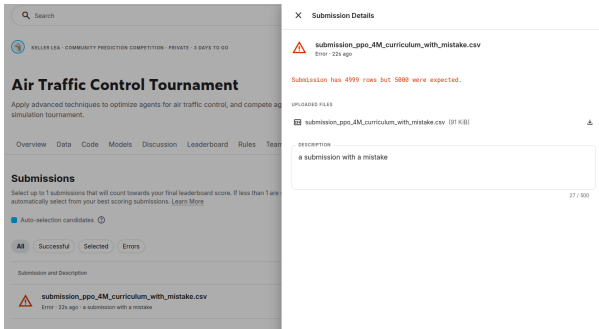
⚠ Do not read the magnitude

Kaggle sorts one float, so the four ranked objectives are **packed** into it: a difference in failures dominates any possible difference in congestion, which dominates the 4D gate, which dominates clearances. **Only the ordering means anything.** Aircraft touched does not fit; it breaks ties only on our board.

>_ Unpack a number off the board

```
python -m competition.run decode 40026885486301
# {"failed": 10, "congestion": 176,
#   "exit_miss_bucket": 26, "clearances": 2089}
```

When Kaggle rejects your file — read the message



The screenshot shows the Kaggle interface for the 'Air Traffic Control Tournament'. On the left, the 'Submissions' section shows a list of submissions, with one highlighted: 'submission_ppo_4M_curriculum_with_mistake.csv' with an error icon and the message 'Error - 22s ago - a submission with a mistake'. On the right, the 'Submission Details' panel shows the same submission with a red error message: 'Submission has 4999 rows but 5000 were expected.' Below this, the 'UPLOADED FILES' section shows the file 'submission_ppo_4M_curriculum_with_mistake.csv' (91 KiB). The 'DESCRIPTION' section shows the text 'a submission with a mistake'.

A truncated submission, refused with something you can act on

Every seed carries all 50 rows

Most episodes end early — the brick ends nearly all of them in a mid-air inside a dozen steps. The rows past that point are written as the **idle** action, so the file covers the whole scored horizon. `submit` does this for you.

💡 The errors are written for you, not for us

Every rejection the metric can raise names the fix — “*Submission is missing 3000 of 5000 required rows. Rebuild it with `python -m competition.run submit`.*” If you edited the file by hand, that is why.

The private board

The zip: assemble it, describe it, and what happens to it

> sessions/03-advanced — four commands

```
mkdir -p my-team/model
cp airtraffic/runs/mine/final_model.zip my-team/model/policy.zip
cp -r airtraffic/envs my-team/envs      # only if you changed it
(cd my-team && zip -r ../my-team.zip .)  # after writing the yaml
```

 **my-team/submission.yaml** — written by hand; team is the only required key

```
team: my-team
algo: PPO                                     # recorded only
env_entry_point: envs.flight_4d:Flight4DEnv  # omit if no envs/
deterministic: true                         # false = sampled
notes: "what you changed, in one line"
```

Edited `flight_4d.py` in place: the entry point is exactly that.

Subclassed into `envs/my_env.py`: name that class. Hand-written

controller: `policy/ + policy_entry_point:`

`policy.my_policy:MyPolicy, no model/.`

 **my-team.zip** — what the commands produce

```
submission.yaml      # required
model/policy.zip     # required, unless policy_entry_point
envs/                # only if changed -- the WHOLE package
policy/              # only with policy_entry_point
```

What we do with it

1. Unzip, read the yaml. **2.** A named `env_entry_point` must pass the **contract suite** — red is unscored, no manual fixes. **3.** 100 episodes on the secret seeds, the same rank key — **two environments in lockstep**: yours produces the observations, ours is stepped with the same action and **every KPI is read from ours**, loaded by file path so nothing you ship can stand in for it.

Three ways to be rejected, all avoidable

⚠ 1. Ship the whole `envs/` package

Your `envs/` replaces ours on the import path, so from `envs.flight_4d` `import Flight4DEnv` looks inside *your* package and fails. Copy the directory, add your module beside the others, import **relatively**:

```
# envs/my_env.py
from .flight_4d import Flight4DEnv
class MyFlightEnv(Flight4DEnv): ...
```

The single most likely rejection, by far.

⚠ 2. `policy_entry_point` takes no arguments

A hand-written controller is a legitimate entry — any class with SB3's `predict(obs, deterministic)`, instantiated bare. Read the scenario from `competition.config.SCENARIO`. Two worked examples are in the Drive folder.

⚠ 3. `vecnormalize.pkl` is not applied

`Flight4DEnv` already emits observations in $[-1, 1]$, so the harness does not wrap the scored environment. Trained under `VecNormalize`? It warns rather than ignores — tell us.

Run the game yourself — and, if you changed the environment, get onto Kaggle

>_ The exact check the organiser runs, on your own public-seed numbers

```
cd sessions/03-advanced
python -m competition.ingest --zip my-team.zip --contract          # green = it WILL be scored
python -m competition.ingest --zip my-team.zip --contract --limit 10 # ten-second smoke test
python -m competition.ingest --zip my-team.zip --contract --out submission.csv # + Kaggle file
```

⚠ submit only knows the stock environment

A policy trained on observations of *your* environment would be fed the wrong vector.
ingest --out flies your zip exactly as the private board does — yours observed, ours scored, in lockstep — on the public seeds, and writes the trace. Then replay and upload as usual.

Just the contract suite, if you only changed the env

```
cd sessions/03-advanced/airtraffic
RLB_ENV_ENTRY_POINT=envs.flight_4d:Flight4DEnv \
python -m pytest tests/test_env_contract.py -q # 24 tests, <1 s
```

Spaces, reproducible `reset(seed=)`, same seed
+ same actions $\Rightarrow$ same trajectory, the KPI keys
in info, termination inside the horizon.

The tools while you iterate

train_4d.py — the flags you will actually touch

> From sessions/03-advanced/airtraffic

```
# the sprint budget (~4 min), then the reference run (~80 min)
OMP_NUM_THREADS=1 python scripts/train_4d.py --steps 2000000 --out runs/mine
OMP_NUM_THREADS=1 python scripts/train_4d.py --steps 40000000 --out runs/ref \
    --target-kl 0.05 --curriculum --curriculum-frac 0.2 \
    --auto-entropy --auto-entropy-target 0.35 --checkpoint-every 500000
```

Flag	Default	What it does
--steps / --out / --seed	1.5M / — / 123	budget, run directory, seed
--n-envs	8	parallel environments; 16 measured <i>slower</i> than 8 here
--idle-bias	2.5	prior mass on idling at initialisation; 0 ablates it
--hold-steps, --w-safe, --w-clearance	env defaults	re-tune the environment's own constants
--curriculum, --curriculum-frac	off, 0.2	pin $n=2$, widen to $U(2, 4)$ over this fraction of training
--auto-entropy, --auto-entropy-target	off, 0.35	tune <code>ent_coef</code> so entropy tracks a fraction of $\ln n_{\text{actions}}$
--checkpoint-every	500k	a checkpoint this often; 0 disables

⚠ OMP_NUM_THREADS=1 is not optional

Torch otherwise opens ≈ 46 threads per process; two jobs on 16 cores measured **31–39 fps against 428** pinned. It looks exactly like a slow environment and is not. `--help` lists the rest.

score_4d.py, calibrate_4d.py, render_4d.py

>_ Score — 100 held-out seeds, lexicographic, [det] and [sto] for every checkpoint

```
python scripts/score_4d.py --agent noop --agent random --agent rule-based --agent
runs/mine/final_model.zip
# --seeds 100 --seed0 900000 --hold-steps 5 (must match training)
```

>_ Calibrate — what each reward term is worth, before you spend a run on it

```
python scripts/calibrate_4d.py --seeds 40 --agent noop --agent random --agent rule-based # <1
min
```

>_ Render — an episode for the projector, or the action vocabulary one clip at a time

```
python scripts/render_4d.py --agent rule-based --agent runs/mine/final_model.zip --seed 900000
python scripts/render_4d.py --n-flights 2 --n-range 2 2 --seed 2 --agent noop --agent fl_inc2_t --
ext gif
```

Two scorers, one environment. score_4d.py while you iterate; competition.run to package and replay a submission. They score **different seed blocks** — quote the seed list with any number.

Head to head, and the post-mortem passes

>_ While someone is at your elbow: 20 seeds, seconds

```
cd sessions/03-advanced
```

```
python -m competition.run --limit 20 compare --agent rule-based --agent <zip>
```

```
python -m competition.run postmortem --agent <zip> # every pass below, on the public seeds
```

Pass	The question it asks
do-nothing	Does it beat the brick?
stochastic	How much survives sampling, not <code>argmax</code> ?
next-best	Swap every decision for the runner-up.
ϵ-random	Corrupt actions with $\epsilon \in \{0.05, 0.1, 0.25\}$.
best-extraction	k rollouts per seed, keep the best.

Noise can flatter you

Perturbing the **brick** at $\epsilon=0.25$ drops its congestion 752 $\rightarrow$ 670 and failures 100 $\rightarrow$ 84. “Safer with noise” is not evidence of robustness — read failures alongside congestion.

Trophies — several ways to win

Trophy	Milestone	Decided by
Golden Holding Pattern	best deterministic score over the 100 private seeds	the rank key
Zero-Conflict Wings	no loss of separation on all 100 seeds	the KPIs
Minimal-Intervention	safe with the fewest clearances	the KPIs
Iron Stomach	smallest deterministic↔stochastic gap (opt-in)	the post-mortem
The Brick with Wings	beat the do-nothing baseline — the participation bar	the post-mortem
Black Box Award	best post-mortem / QA writeup	jury
Icarus Award	boldest redesign that crashed and burned	jury

💡 Why the last two are not computed

They are jury calls, and pretending a metric decides them would be worse than admitting a human does.

⚠️ Why Iron Stomach excludes heuristics

A heuristic's stochastic gap is zero because it has **nothing to sample from**. A robustness trophy must not go to something that was never perturbed.

Deliverables, and the clock

The sprint, in pairs

15 pick a track; read the part of `envs/flight_4d.py` it touches

30 implement — one drives, one keeps the baseline honest

15 train 200k steps; score against the baselines

10 present: what changed, what improved, what broke

The KPI table is the deliverable

Not the reward curve. A return without the KPIs, the episode length and a baseline beside it has not reported anything. The design note is graded on the hypothesis, not the result.

Command cheat sheet

Environment —

sessions/03-advanced/airtraffic/

```
# the contract: before you change anything, and after
pytest

# train on the sprint budget (~4 min)
OMP_NUM_THREADS=1 python scripts/train_4d.py \
    --steps 200000 --out runs/mine --seed 123
python scripts/train_4d.py --idle-bias 0          # no idle prior
python scripts/train_4d.py --w-clearance 0.05    # cheaper clearances
python scripts/train_4d.py --hold-steps 8        # longer manoeuvres

# score on 100 held-out seeds, ALWAYS beside the baselines
python scripts/score_4d.py --agent noop --agent random \
    --agent rule-based --agent runs/mine/final_model.zip

# what each reward term is worth -- BEFORE a run, not after
python scripts/calibrate_4d.py
# one episode for the projector -> renders/*.mp4
python scripts/render_4d.py --agent runs/mine/final_model.zip
tensorboard --logdir runs/          # curves; score the KPIs
```

Competition harness — sessions/03-advanced/

```
# -- public board (Kaggle) -----
# 1. roll out on the 100 public seeds; prints your score
python -m competition.run submit --agent <zip> --out submission.csv
# 2. re-fly the file exactly as Kaggle will; must match 1.
python -m competition.run replay --submission submission.csv
# 3. upload submission.csv on the competition page
# a board number -> failed / congestion / exit_miss / clear.
python -m competition.run decode <number off the board>
# head to head in seconds; writes nothing
python -m competition.run --limit 20 compare \
    --agent rule-based --agent <zip>
# every robustness pass; 2+ agents -> the trophies too
python -m competition.run postmortem --agent rule-based --agent <zip>

# -- private board (the zip) -----
# the organiser's exact check, public seeds: green = scored
python -m competition.ingest --zip <team>.zip --contract
# changed the environment? your ONLY route to Kaggle
python -m competition.ingest --zip <team>.zip --contract \
    --out submission.csv
```

Handbook — docs/advanced/kaggle.md · competition.md ·

training.md · evaluation.md · docs/reference/tooling.md

Links kaggle.com/t/a0a74b5818114812a2b076b172dfe197 ·

drive.google.com/drive/folders/1Mrv5aZ70LDN5goMj0k67XFGK84LZtBOI

Measured wall-clock, one 16-core laptop, no GPU

What	Command	Wall-clock
environment contract tests	<code>pytest</code>	< 30 s
scoring + Kaggle metric tests	<code>pytest (in sessions/03-advanced)</code>	< 30 s
design-sprint budget (200k steps)	<code>train_4d.py --steps 200000</code>	≈ 4 min
full reference run (4M steps)	<code>train_4d.py --curriculum --auto-entropy</code>	≈ 80 min
board, 100 seeds × 2 baselines	<code>score_4d.py</code>	≈ 10 s
board, + rule-based + a checkpoint	<code>score_4d.py --agent <zip></code>	≈ 2 min
reward-term calibration	<code>calibrate_4d.py</code>	< 1 min
build a submission, 100 seeds	<code>competition.run submit</code>	≈ 2 min
replay a submission (numpy only)	<code>competition.run replay</code>	< 1 min
validate a zip, contract + public seeds	<code>competition.ingest --contract</code>	≈ 3 min
render one episode to mp4	<code>render_4d.py</code>	≈ 20 s

Measure your own machine before you budget the sprint. A 200k-step run says whether a design is *alive*, not whether it is good — say which one you think you have.

Submit the rule-based agent in the next two minutes.

Then improve it, as often as you like.